HS PF

Programmieren in Python

Python-Grundlagen: Numpy Teil 4

Prof. Dr.-Ing. Stefan Kray

Mathematische Operationen

- NumPy unterstützt viele mathematische Operationen auf Arrays

```
np.sin(x), np.cos(x), np.tan(x), np.arccos(x), np.arccos(y), ...
np.sqrt(x), np.log(x), np.log10(x)
np.mean(x), np.sum(x), np.cumsum(x)
```

- Array runden:

```
x=np.array([0.,1.33333333,2.66666667,4.])
np.ceil(x) -> array([0., 2., 3., 4.])
np.floor(x) -> array([0., 1., 2., 4.])
np.round(x) -> array([0., 1., 3., 4.])
np.trunc(x) -> array([0., 1., 2., 4.])
np.clip(x,1,2) -> array([1.,1.33333333,2.,2.]) #abschneiden bei <-1 und >2
```

- FFT:

```
np.fft.fft(x) -> array([ 8.        +0.j        , -2.66666667+2.66666667j,
                        -2.66666667+0.j        , -2.66666667-2.66666667j])
y=np.fft.ifft(x) -> array([ 2.        +0.j        , -0.66666667-0.66666667j,
                           -0.66666667+0.j        , -0.66666667+0.66666667j])
np.fft.fftshift(y) -> array([-0.66666667+0.j        , -0.66666667+0.66666667j,
                             2.        +0.j        , -0.66666667-0.66666667j])
```

Polyfit

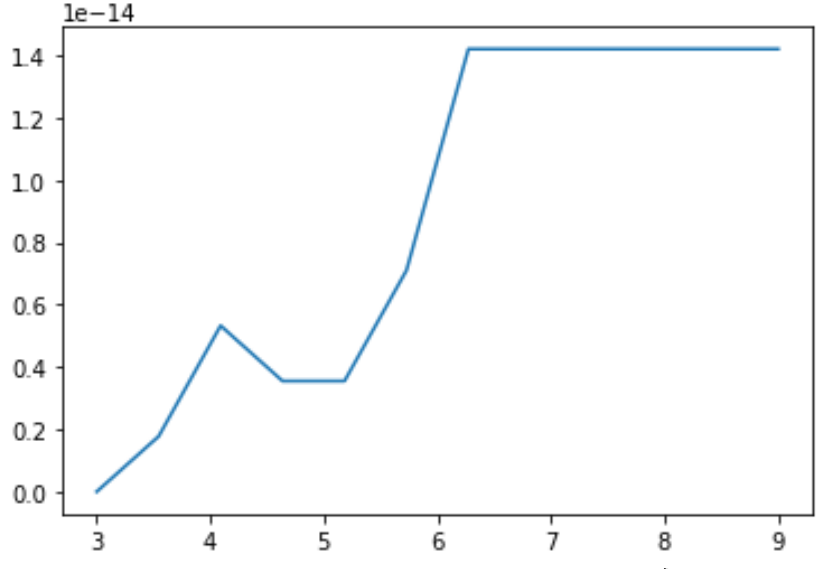
- NumPy verfügt über Funktionen zum Fitten von Daten
- numpy.polyfit(x, y, deg) → x,y: Arrays, deg: Polynomordnung
- Fit a polynomial p(x) = p[0] * x**deg + ... + p[deg] of degree deg to points (x, y). Returns a vector of coefficients p that minimises the squared error in the order deg, deg-1, … 0.

```
x=np.linspace(3,9,12)
y=x*x-3
p=np.polyfit(x,y,2)
p -> np.array([ 1.00000000e+00, -3.04035618e-15, -3.00000000e+00])
```

entspricht ca.: $1 * x^2 + 0 * x^1 - 3 * x^0 = x^2 - 3$

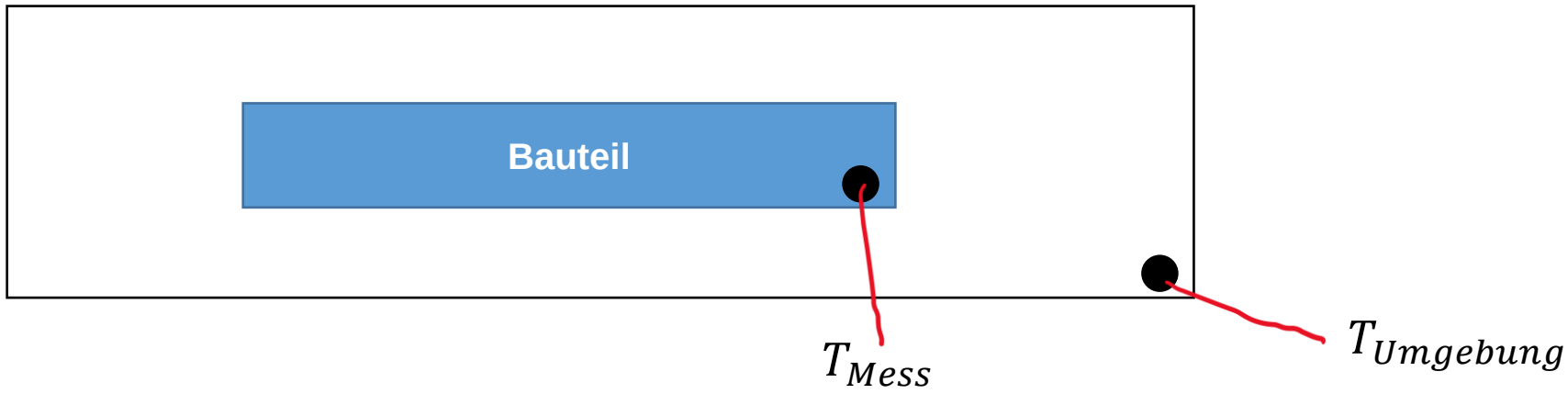
```
fit=p[0]*x**2+p[1]*x+p[2]
plt.plot(x,y-fit)
```

Fitfehler



Aufgabe fit.py

- Bei einer Temperaturmessung soll die Bauteiltemperatur erfasst werden. Dazu wird ein Temperatursensor am Bauteil angebracht. Ein Umgebungssensor misst die Temperatur des Raumes.



- Es wird ein lineares Modell angenommen: $T_{Mess} = T_{Bauteil} + T_{Umgebung}$
- Die Messtemperatur ist in der Datei „Messtemperatur.txt“ abgelegt
- Die Umgebungstemperatur ist in der Datei „Umgebungstemperatur.txt“ abgelegt
- Errechnen Sie die Bauteiltemperatur
- Verwenden Sie np.loadtxt und np.polyfit

Vektoren

- Vektoren in Numpy als Arrays darstellen

```
x=np.array([1,0,0])
y=np.array([0,1,0])
```

- Vektormathematik:
  - Addition / Subtraktion: x+y, x-y
  - Skalar addieren, multiplizieren: x+3, x*3, …
  - Skalarprodukt: `np.dot(x,y)` → 0
  - Vektorprodukt: `np.cross(x,y)` → array([0, 0, 1])
  - Vektorlänge: `np.linalg.norm(x)` → 1

Matrizen (1)

- Matrizen werden als normale 2D-Arrays dargestellt

```
a=np.array([[1,2],[3,4]])
b=np.array([[5,6],[7,8]])
```

- Addition / Subtraktion wie bisher

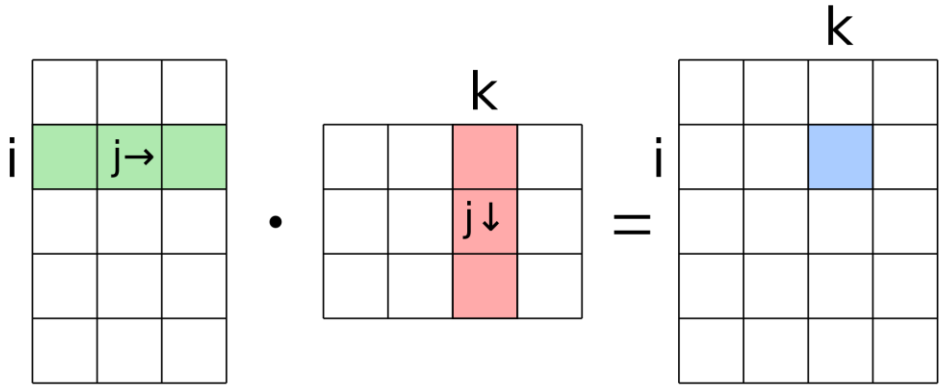
```
a+b -> array([[ 6,  8],
              [10, 12]])
```

- Matrixmultiplikation

```
np.matmul(a,b) -> array([[19, 22],
                         [43, 50]])
```

-

  Kurzform: a@b



Von Quartl - Eigenes Werk, CC BY-SA 3.0,
https://commons.wikimedia.org/w/index.php?curid=27634998

Matrizen (2)

- Eigenwerte: $Ax = \lambda x$
- `lam,x=np.linalg.eig(a)`

```
lam -> np.array([-0.37228132,  5.37228132])
x  -> np.array([[-0.82456484, -0.41597356],
                [ 0.56576746, -0.90937671]])
```

Beweis:

```
np.matmul(a,x[:,0]) -> array([ 0.30697009, -0.21062466])
lam[0]*x[:,0] -> array([ 0.30697009, -0.21062466])
```

Matrizen (3)

- Determinante:
  - $A = \begin{pmatrix} a & c \\ b & d \end{pmatrix} \rightarrow \det A = \begin{vmatrix} a & c \\ b & d \end{vmatrix} = ad - bc$
- np.linalg.det(a) → -2
- Beweis: $1 * 4 - 3 * 2 = -2$

Matrizen und Gleichungssysteme

- Beispiel: Gleichungssystem lösen:
  - $x_0 + 2x_1 = 1$ und $3x_0 + 5x_1 = 2$
  - Als Matrix darstellen:

```
a = np.array([[1, 2], [3, 5]])
b = np.array([1, 2])
np.linalg.solve(a, b) -> array([-1.,  1.])
```

Beweis: $-1 + 2 * 1 = 1$ und $3 * -1 + 5 * 1 = 2$

$1x_0 + 2x_1 = 1$

$3x_0 + 5x_1 = 2$

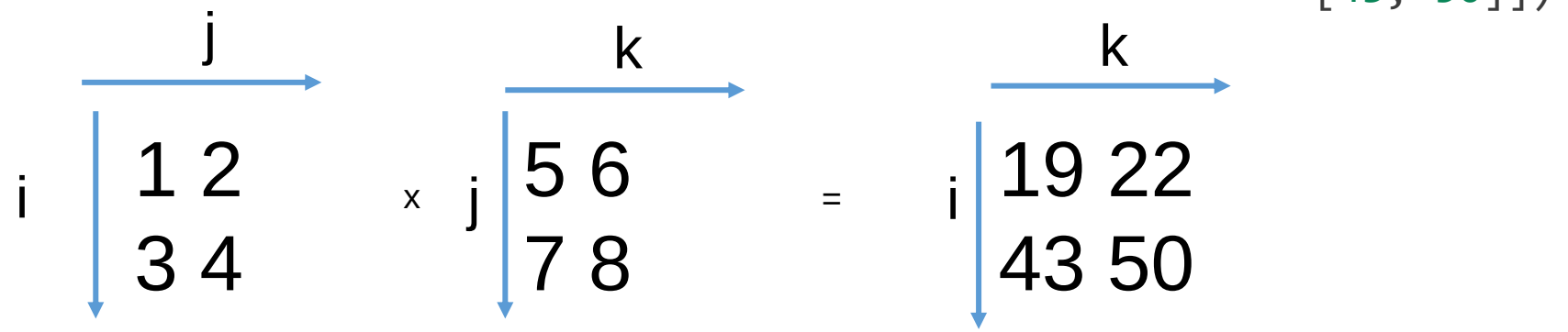
$a = \begin{pmatrix} 1 & 2 \\ 3 & 5 \end{pmatrix}$ $b = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$

Einsum (1)

- NumPy „Einsteinsumme“ (Einsteinsche Summenkonvention)
- https://de.wikipedia.org/wiki/Einsteinsche_Summenkonvention
- Multipliziert zwei Matrizen beliebig miteinander
- Beispiel:

```
a=np.array([[1,2],[3,4]])
b=np.array([[5,6],[7,8]])
```

Matrixmultiplikation: `np.einsum('ij,jk->ik',a,b) -> array([[19, 22],`
`[43, 50]])`



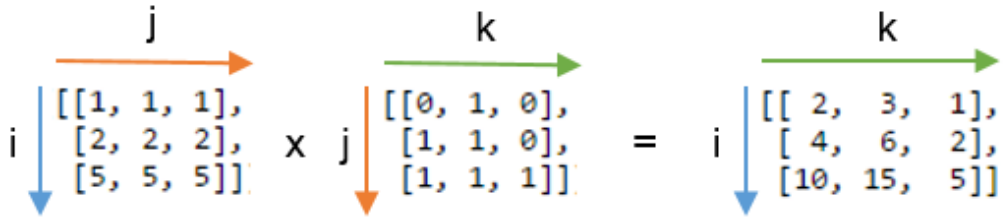
Einsum kommt nicht in der Klausur vor

Einsum (2)

- NumPy „Einsteinsumme“
- Beispiel:

```
np.einsum('ij,jk->ik',a,b)
```

Regeln:
https://ajcr.net/Basic-guide-to-einsum/



To understand how the output array is calculated, remember these three rules:

- **Repeating letters between input arrays means that values along those axes will be multiplied together. The products make up the values for the output array.**

In this case, we used the letter `j` twice: once for `A` and once for `B`. This means that we're multiplying each row of `A` with each column of `B`. This will only work if the axis labelled by `j` is the same length in both arrays (or the length is 1 in either array).

- **Omitting a letter from the output means that values along that axis will be summed.**

Here, `j` is not included among the labels for the output array. Leaving it out sums along the axis and explicitly reduces the number of dimensions in the final array by 1. Had the output signature been `'ijk'` we would have ended up with a 3x3x3 array of products. (And if we gave *no* output labels but just write the arrow, we'd simply sum the whole array.)

- **We can return the unsummed axes in any order we like.**

If we leave out the arrow `'->'`, NumPy will take the labels that appeared once and arrange them in alphabetical order (so in fact `'ij,jk->ik'` is equivalent to just `'ij,jk'`). If we want to control what our output looked like we can choose the order of the output labels ourself. For example, `'ij,jk->ki'` delivers the transpose of the matrix multiplication (notice that `k` and `i` were switched in the output labelling).

Einsum (3)

- NumPy „Einsteinsumme“
- Regeln:
  https://ajcr.net/Basic-guide-to-einsum/

Let `A` and `B` be two 1D arrays of compatible shapes (meaning the lengths of the axes we pair together either equal, or one of them has length 1):

Call signature	NumPy equivalent	Description
`('i', A)`	`A`	returns a view of A
`('i->', A)`	`sum(A)`	sums the values of A
`('i,i->i', A, B)`	`A * B`	element-wise multiplication of A and B
`('i,i', A, B)`	`inner(A, B)`	inner product of A and B
`('i,j->ij', A, B)`	`outer(A, B)`	outer product of A and B

Einsum (4)

- NumPy „Einsteinsumme“
- Regeln:
  https://ajcr.net/Basic-guide-to-einsum/

Now let `A` and `B` be two 2D arrays with compatible shapes:

Call signature	NumPy equivalent	Description
`('ij', A)`	`A`	returns a view of A
`('ji', A)`	`A.T`	view transpose of A
`('ii->i', A)`	`diag(A)`	view main diagonal of A
`('ii', A)`	`trace(A)`	sums main diagonal of A
`('ij->', A)`	`sum(A)`	sums the values of A
`('ij->j', A)`	`sum(A, axis=0)`	sum down the columns of A (across rows)
`('ij->i', A)`	`sum(A, axis=1)`	sum horizontally along the rows of A
`('ij,ij->ij', A, B)`	`A * B`	element-wise multiplication of A and B
`('ij,ji->ij', A, B)`	`A * B.T`	element-wise multiplication of A and B.T
`('ij,jk', A, B)`	`dot(A, B)`	matrix multiplication of A and B
`('ij,kj->ik', A, B)`	`inner(A, B)`	inner product of A and B
`('ij,kj->ikj', A, B)`	`A[:, None] * B`	each row of A multiplied by B
`('ij,kl->ijkl', A, B)`	`A[:, :, None, None] * B`	each value of A multiplied by B